

Wymagania minimalne

R1 Backend API

Pole	Treść
Decyzja	REST API (Express.js) jako warstwa API, udokumentowana za pomocą Swagger.
Kontekst	System posiada 4 kluczowe, powiązane relacjami zasoby (Użytkownicy, Filmy, Seanse, Rezerwacje). Frontend musi w sposób bezpieczny i autoryzowany zarządzać tymi zasobami, a system potrzebuje przejrzystego API.
Alternatywy	GraphQL (eliminuje over-fetching), tRPC (pełne end-to-end type safety w TS), gRPC (bardzo szybka, binarna komunikacja)
Uzasadnienie	REST najbardziej odpowiada charakterystyce systemu, którego funkcjonalność opiera się na zarządzaniu zasobami. Ta architektura pozwala wykorzystać standardowe mechanizmy protokołu HTTP, upraszcza implementację autoryzacji oraz umożliwia wygenerowanie interaktywnej dokumentacji API za pomocą Swagger. W porównaniu z GraphQL rozwiązanie jest prostsze w implementacji i utrzymaniu, a jednocześnie w pełni spełnia wymagania projektu.
Trade-offy	Nie uzyskujemy e2e type safety pomiędzy frontendem, a backendem (w przeciwieństwie do tRPC), trzeba pamiętać o osobnej aktualizacji dokumentacji (swagger.ts) i schematów walidacji (Zod) (to świadomie zaakceptowany trade-off). W przypadku konieczności bardziej skomplikowanych zapytań konieczność utworzenia nowych endpointów (w przeciwieństwie do GraphQL).

R2 Baza Danych

Pole	Treść
Decyzja	PostgreSQL jako persystentna, relacyjna baza danych.
Kontekst	Aplikacja musi trwale przechowywać, przetwarzać i wiązać ze sobą kluczowe dane serwisu (konta użytkowników, katalog filmów, zajęte miejsca, harmonogram seansów). Ponadto system wymaga wysokiej spójności danych i bezpiecznej obsługi współbieżnych operacji (żeby nie sprzedać tego samego miejsca wielokrotnie).
Alternatywy	MongoDB/NoSQL (elastyczny schemat, łatwiejsze skalowanie horyzontalne), MySQL (dojrzała relacyjna baza danych o podobnych możliwościach, jednak PostgreSQL oferuje bogatszy zestaw funkcji SQL, bardziej rozbudowane typy danych oraz zaawansowane mechanizmy związane z integralnością danych)
Uzasadnienie	Rezerwacje kinowe to klasyczny problem silnie relacyjny. PostgreSQL zapewnia pełną zgodność z modelem ACID, co gwarantuje bezpieczeństwo transakcji przy procesowaniu biletów (blokowanie i opłacanie miejsc). Bazy dokumentowe nie oferują w tym przypadku istotnych korzyści, ponieważ model danych jest silnie ustrukturyzowany i opiera się na relacjach między encjami.
Trade-offy	Relacyjny model danych wymaga zarządzania schematem bazy oraz wykonywania migracji przy zmianach struktury danych. W porównaniu z bazami dokumentowymi ogranicza to elastyczność wprowadzania zmian. Dodatkowo skalowanie horyzontalne systemów relacyjnych jest zazwyczaj bardziej złożone niż w przypadku wielu rozwiązań NoSQL.

Pole	Treść
Decyzja	Prisma jako warstwa ORM i narzędzie do zarządzania schematem i migracjami.
Kontekst	Mając bazę PostgreSQL oraz backend napisany w środowisku TS, aplikacja potrzebuje warstwy abstrakcji do bezpiecznej komunikacji z bazą. Niezbędne jest narzędzie, które zagwarantuje bezpieczeństwo typów oraz ustandaryzowany, zautomatyzowany sposób wersjonowania struktury bazy (migracje).
Alternatywy	TypeORM (popularny, oparty na klasach i dekoratorach, jednak oferujący mniej przewidywalne wsparcie dla zaawansowanego typowania w TS), Drizzle ORM (lekki, bliski czystemu SQL, jednak z mniejszym ekosystemem gotowych narzędzi), bezpośrednie pisanie zapytań SQL z użyciem query buildera np. Knex (maksymalna wydajność i kontrola, ale brak automatycznego mapowania typów).
Uzasadnienie	Prisma bardzo dobrze integruje się z ekosystemem TypeScript. Na podstawie deklaratywnego schematu (schema.prisma) generuje klienta bazodanowego z typami zgodnymi ze strukturą danych, co ogranicza ryzyko rozbieżności pomiędzy schematem bazy a kodem aplikacji. Dodatkowo Prisma zapewnia wbudowany mechanizm migracji. W porównaniu z TypeORM oferuje bardziej przewidywalny model definiowania schematu i lepszą integrację z TypeScript, natomiast względem Drizzle zapewnia bogatszy ekosystem narzędzi i większą automatyzację pracy z bazą danych.
Trade-offy	Zastosowanie ORM wprowadza dodatkową warstwę abstrakcji pomiędzy aplikacją a bazą danych, co może utrudniać analizę wydajności oraz prowadzić do generowania mniej optymalnych zapytań SQL w bardziej złożonych przypadkach. Dodatkowo Prisma wymaga generowania klienta po zmianach schematu oraz zużywa więcej zasobów niż lżejsze rozwiązania, takie jak Drizzle czy bezpośrednie wykorzystanie query builderów.

R3 Frontend

Pole	Treść
Decyzja	Architektura SPA oparta na bibliotece React i narzędziu Vite .
Kontekst	Aplikacja składa się z dynamicznych i wysoce interaktywnych widoków (np. mapa wyboru miejsc na sali, panel administratora, płatności). Aplikacja musi umożliwić użytkownikowi interakcję z systemem w sposób płynny, bez przeładowania całej strony przy każdym przejściu między ekranami.
Alternatywy	Next.js / SSR (szybki czas pierwszego renderowania, ale większa złożoność infrastruktury), HTMX (trudniejsze przy budowie interaktywnych widoków sali kinowej), Angular / Vue (inne frameworki SPA o podobnych możliwościach, jednak React oferuje prostszą integrację z wybranym stosem technologicznym i szeroką dostępność gotowych bibliotek wspierających budowę interfejsów).
Uzasadnienie	Aplikacja opiera się na interaktywnych widokach, takich jak wybór miejsc w sali kinowej, panel administracyjny oraz proces rezerwacji i płatności. W tego typu scenariuszach kluczowe znaczenie ma płynne zarządzanie stanem interfejsu oraz szybka reakcja UI na działania użytkownika bez konieczności przeładowywania całej strony. Architektura SPA umożliwia utrzymanie stanu aplikacji po stronie klienta oraz komunikację z backendem poprzez API, co zapewnia wysoką responsywność i spójne doświadczenie użytkownika. Z tego względu wybrano podejście SPA oparte na React, które dobrze wspiera budowę dynamicznych, komponentowych interfejsów oraz wyraźny podział między warstwą frontendową a backendową.
Trade-offy	W modelu SPA przeglądarka musi pobrać cały szkielet JavaScriptu, zanim wyrenderuje aplikację, co wydłuża initial payload w porównaniu do SSR.

Pole	Treść
Decyzja	Tailwind CSS v4 jako framework stylowania z systemem motywów opartym na CSS variables i ThemeContext.
Kontekst	Aplikacja wymaga spójnego systemu wizualnego oraz oferuje obsługę motywu jasnego i ciemnego zgodnie z preferencjami użytkownika.
Alternatywy	CSS Modules (izolacja stylów per komponent, ale brak gotowych klas użytkowych i konieczność ręcznego budowania systemu motywów), styled-components (CSS-in-JS, dynamiczne style, ale większy narzut runtime i gorsza integracja z Vite), czysty CSS (pełna kontrola, ale duży nakład pracy).
Uzasadnienie	Tailwind eliminuje konieczność pisania własnych klas CSS dla typowych wzorców layoutu i typografii. Motywy zaimplementowano przez CSS variables przełączane klasą .dark na document.documentElement - podejście natywne, bez dodatkowych zależności. ThemeContext zarządza stanem motywu po stronie React i przechowuje wybór użytkownika w localStorage. Wartości zmiennych są mapowane na klasy Tailwinda przez @theme w index.css, co pozwala używać klas takich jak bg-page czy text-accent w komponentach.
Trade-offy	Tailwind v4 jest nową wersją z odmienną konfiguracją względem v3 (plugin Vite zamiast PostCSS), co ogranicza dostępność dokumentacji i przykładów. Klasy użytkowe w JSX mogą być trudniejsze w czytaniu przy złożonych komponentach.

Pole	Treść
Decyzja	React Context API jako mechanizm globalnego stanu frontendowego, podzielony na trzy niezależne konteksty: AuthContext, NotificationContext, ThemeContext.
Kontekst	Aplikacja wymaga globalnie dostępnego stanu sesji użytkownika, systemu powiadomień i motywu. Stan ten musi być dostępny w wielu niezależnych komponentach.
Alternatywy	Zustand (lekki globalny store, minimalizuje rerenderowanie, dobry dla stanu dzielonego między wieloma komponentami niezwiązanymi hierarchicznie), Redux Toolkit (rozbudowany, z DevTools, ale nadmiarowy dla tej skali projektu), jeden duży Context (anty wzorec)
Uzasadnienie	Aplikacja posiada trzy rodzaje globalnego stanu: sesję użytkownika, powiadomienia i motyw. Każdy z nich zmienia się rzadko i jest konsumowany przez niewielu, przewidywalnych konsumentów. Podział na trzy niezależne konteksty ogranicza zasięg rerenderów: zmiana powiadomienia nie rerenderuje komponentów czytających stan auth.
Trade-offy	Context rerenderuje wszystkich konsumentów danego kontekstu przy każdej zmianie wartości, nawet jeśli konkretny komponent używa tylko podzbioru danych. Przy rosnącej aplikacji i większej liczbie konsumentów Zustand byłby lepszym wyborem.

R4 Autentykacja

Pole	Treść
Decyzja	Bezstanowa autoryzacja oparta na tokenach JWT przechowywanych w ciasteczkach HttpOnly z mechanizmem kontroli ról (RBAC) i hashowanie haseł za pomocą bcrypt .
Kontekst	Aplikacja musi w bezpieczny sposób weryfikować tożsamość użytkowników oraz odróżniać zwykłych klientów od administratorów. Niezbędna jest ochrona wybranych zasobów po stronie API oraz ukrywanie chronionych widoków na frontendzie.
Alternatywy	JWT w local storage (podatne na XSS), zewnętrzni dostawcy Oauth2 (zależność od zewnętrznych usług), sesje stanowe (nie zgodne z bezstanowością API)
Uzasadnienie	Wybrano JWT przechowywane w ciasteczkach HttpOnly jako kompromis pomiędzy bezpieczeństwem a bezstanową architekturą API. Flaga HttpOnly ogranicza ryzyko kradzieży tokena w przypadku podatności XSS, eliminując konieczność przechowywania go w localStorage. Sesja użytkownika jest odtwarzana przy każdym załadowaniu aplikacji przez odpytanie endpointu /api/auth/me (funkcja refreshUser), co gwarantuje zawsze aktualny stan uprawnień. Rzeczywista autoryzacja i ochrona zasobów egzekwowana jest po stronie API przez middleware authenticateToken i requireAdmin, a po stronie frontendu komponent ProtectedRoute steruje dostępem do widoków na podstawie roli. Hasła są hashowane algorytmem bcrypt z solą. Mechanizm RBAC rozróżnia role USER i ADMIN na poziomie zarówno API jak i interfejsu.
Trade-offy	Przechowywanie tokenów w ciasteczkach HttpOnly wymaga dodatkowej konfiguracji CORS oraz ustawienia credentials:true po stronie klienta i serwera. W porównaniu do sesji stanowych, JWT jest trudniejszy do natychmiastowego unieważnienia bez zastosowania dodatkowych mechanizmów (np. blacklisty lub rotacji tokenów).

R5 Konteneryzacja

Pole	Treść
Decyzja	Wykorzystanie Dockera i Docker Compose do konteneryzacji całej aplikacji (baza danych, backend, frontend).
Kontekst	Aplikacja składa się z trzech niezależnych warstw (baza danych PostgreSQL, backend Node.js, frontend React). Aby ułatwić uruchomienie i konfigurację aplikacji Docker zapewnia środowisko, które można podnieść jednym poleceniem na każdej maszynie z Dockerem.
Alternatywy	Ręczna, lokalna instalacja i konfiguracja stosu technologicznego (czasochłonne, inna konfiguracja na różnych systemach, możliwość wystąpienia błędów i konfliktów), Kubernetes (nadmiarowe rozwiązanie w stosunku do skali aplikacji)
Uzasadnienie	Docker Compose rozwiązuje problem powtarzalności środowiska oraz upraszcza proces uruchamiania całego systemu, umożliwiając uruchomienie wszystkich usług jednym poleceniem. Wbudowana sieć Dockera pozwala na komunikację między usługami. Dodatkowo, mechanizm health check pozwala na kontrolę gotowości usług i ogranicza ryzyko uruchomienia backendu przed pełną inicjalizacją bazy danych.
Trade-offy	Konteneryzacja wprowadza dodatkową warstwę abstrakcji. Wymaga również utrzymywania dodatkowych plików konfiguracyjnych (Dockerfile dla każdego serwisu, docker-compose.yml).

R6 Repozytorium

Pole	Treść
Decyzja	Wykorzystanie systemu kontroli wersji Git oraz platformy Github do rozproszonego wersjonowania kodu z dokumentacją w README.md.
Kontekst	Projekt na każdym etapie rozwoju wymaga bezpiecznego śledzenia historii zmian, możliwości cofania się do poprzednich wersji i dostarczenia instrukcji uruchomienia, kodu źródłowego i opisu architektury.
Alternatywy	Gitlab (podobne możliwości), brak systemu kontroli wersji (konieczność trzymania kopii starych wersji projektu)
Uzasadnienie	System kontroli wersji pozwala na powrót do poprzednich wersji projektu w miarę potrzeby, a Github zapewnia wygodny interfejs do interakcji z tym systemem. Umożliwia tworzenie Issues, Pull Requestów, i wykorzystania Github Actions do CI/CD.
Trade-offy	Wykorzystanie Githuba oznacza zależność od zewnętrznego dostawcy i konieczność dostępu do internetu. Publiczne repozytorium wymusza ostrożność nad tym, jakie pliki tam przesyłamy, a co za tym idzie konieczność konfiguracji dodatkowego pliku (.gitignore).

Elementy dodatkowe

CI/CD

Pole	Treść
Decyzja	Wdrożenie CI za pomocą Github Actions w celu automatycznej weryfikacji poprawności i jakości kodu.
Kontekst	Projekt oparty na statycznym typowaniu TypeScripta wymaga poprawnej kompilacji przy każdej zmianie w kodzie.
Alternatywy	Gitlab CI (wymagałoby przeniesienia repozytorium na Gitlab), ręczne uruchamianie buildów, Jenkins (wymaga utrzymania własnego serwera)
Uzasadnienie	Github Actions jest natywnie dostępne w każdym repozytorium Githuba, co umożliwia automatyczne uruchamianie pipelineów przy każdym pushu lub PR. Poprawnie skonfigurowane umożliwia na wyłapanie błędów w konkretnych fragmentach kodu.
Trade-offy	Zależność od Githuba, limity użycia, utrzymanie plików konfiguracyjnych dla każdego serwisu.

Observability

Pole	Treść
Decyzja	Wdrożenie podstawowego mechanizmu observability poprzez wystawienie dedykowanego endpointu /api/health .
Kontekst	Aplikacja składa się z kilku zależnych od siebie nawzajem usług uruchamianych w Docker Compose. Backend jest zależny od dostępności bazy danych, dlatego konieczne jest zapewnienie mechanizmu pozwalającego na sprawdzenie stanu działania aplikacji oraz jej gotowości do obsługi żądań.
Alternatywy	Brak dedykowanego endpointu (utrudnia diagnostykę i kontrolę stanu aplikacji), zewnętrzny monitoring (nadmiarowe dla skali projektu)
Uzasadnienie	Endpoint /api/health umożliwia podstawową weryfikację stanu działania backendu oraz jego zależności, takich jak baza danych. Jest wykorzystywany w środowisku Docker Compose do kontrolowania gotowości usług, co pozwala ograniczyć ryzyko uruchomienia backendu przed pełną inicjalizacją bazy danych. Rozwiązanie to stanowi lekką i wystarczającą formę observability dla potrzeb aplikacji.
Trade-offy	Rozwiązanie zapewnia jedynie podstawowy poziom obserwowalności systemu i nie dostarcza zaawansowanych metryk ani systemu monitoringu. W przypadku rozbudowy systemu konieczne byłoby wdrożenie bardziej zaawansowanych narzędzi obserwacyjnych.

Walidacja danych

Pole	Treść
Decyzja	Wykorzystanie biblioteki Zod do walidacji struktury danych na wejściu API.
Kontekst	Endpointy API przyjmują dane tekstowe od użytkowników. Niepoprawne, złośliwe lub niepełne dane mogłyby doprowadzić do niespójności w bazie danych, błędów działania ORM'a (Prisma) lub podatności bezpieczeństwa.
Alternatywy	Ręczna walidacja oparta na if/else (trudna w utrzymaniu i rozbudowie), Joi (mniejsze wsparcie dla TS).
Uzasadnienie	Zod dobrze integruje się z TS, pozwala na zdefiniowanie schematów, które są następnie używane przez safeParse wewnątrz kontrolerów, co ogranicza ryzyko niepoprawnych danych w bazie.
Trade-offy	Konieczność utrzymywania dodatkowego pliku ze schematami.

Dokumentacja API

Pole	Treść
Decyzja	Dokumentacja API z wykorzystaniem standardu OpenAPI 3.0 i interfejsu Swagger.
Kontekst	API systemu wymaga czytelnej, jednoznacznej i interaktywnej dokumentacji umożliwiającej łatwe testowanie oraz integrację frontend–backend. Ze względu na rozbudowaną strukturę zasobów (Użytkownicy, Filmy, Seanse, Rezerwacje) konieczne jest zapewnienie standardowego kontraktu API.
Alternatywy	Ręczna dokumentacja API (na przykład w Markdownie, brak interfejsu graficznego, brak interaktywnych testów, konieczność ręcznego pisania), Postman (dobre do testowania, ale nie jest formalną specyfikacją API)
Uzasadnienie	Wykorzystanie standardu OpenAPI 3.0 wraz z interfejsem Swagger pozwala na utrzymanie formalnej, strukturalnej specyfikacji API. Dokumentacja jest tworzona i aktualizowana ręcznie, co zapewnia pełną kontrolę nad jej treścią oraz zgodnością z implementacją backendu. Swagger UI umożliwia interaktywne przeglądanie i testowanie endpointów, co ułatwia integrację oraz zrozumienie działania systemu przez frontend.
Trade-offy	Ręczne utrzymywanie specyfikacji OpenAPI wymaga dodatkowej pracy oraz konsekwentnej synchronizacji dokumentacji z implementacją API. Istnieje ryzyko rozbieżności między dokumentacją a kodem, jeśli zmiany w backendzie nie zostaną odpowiednio odzwierciedlone w specyfikacji.

Seed data

Pole	Treść
Decyzja	Wykorzystanie skryptu napisanego w TS zintegrowanego z Prisma do automatycznego wypełnienia bazy danych danymi demonstracyjnymi.
Kontekst	Samo uruchomienie aplikacji tworzy pustą bazę danych, przez co, by zobaczyć pełne możliwości systemu trzeba ręcznie dodawać dane. Aby umożliwić szybkie testowanie i ocenę aplikacji, skrypt dodaje do bazy przykładowe dane.
Alternatywy	Ręczne wpisywanie danych do systemu (czasochłonne, konieczność ponownego wpisywania przy usunięciu wolumenu, czy zmianie maszyny)
Uzasadnienie	Zastosowanie dedykowanego skryptu seed.ts umożliwia dodanie do systemu tych samych danych za każdym razem. Zapewnia to odtwarzalne środowisko, które umożliwia szybkie sprawdzenie funkcjonalności systemu bez konieczności ręcznego przygotowywania danych. Dodatkowo ułatwia testowanie backendu oraz wykrywanie potencjalnych błędów logicznych w aplikacji.
Trade-offy	Dodatkowy plik do utrzymania, wymaga modyfikacji w razie dodania nowych funkcjonalności lub zmiany struktury bazy.